T3000 API & Protocol

Device Monitoring · built Nov 13, 2026

Device Monitoring

Monitor real-time device status, data points, and system health.

Overview

The T3000 monitoring system provides comprehensive visibility into device operation, data collection, and system performance.

Real-Time Dashboard

The main dashboard displays key metrics:

Device Status Panel

- **Connection State**: Online/Offline status
- **Last Communication**: Timestamp of last successful poll
- **CPU Usage**: Device processor utilization
- **Memory Usage**: RAM consumption
- **Uptime**: Time since last restart

Data Point Summary

Quick view of critical points:

- **Inputs**: Sensor readings (temperature, pressure, etc.)
- **Outputs**: Control signals and actuator states
- **Variables**: Calculated values
- **Alarms**: Active alarm count

Data Point Monitoring

Inputs Page

Monitor all input points:

Point	Description	Value	Units	Status	Last Update
IN1	Zone Temperature	72.5	°F	✅ Valid	2s ago
IN2	Supply Pressure	45.2	PSI	✅ Valid	2s ago
IN3	CO2 Level	650	PPM	✅ Valid	2s ago

Features:

- Real-time value updates (2-5 second refresh)
- Color-coded status indicators
- Trend sparklines
- Quick filters and search
- Export to CSV/Excel

Outputs Page

Monitor control outputs:

Point	Description	Command	Feedback	Mode	Override
OUT1	Damper Position	45%	44%	Auto	No
OUT2	Fan Speed	75%	75%	Auto	No
OUT3	Valve Position	0%	0%	Off	Yes

Variables Page

Monitor calculated variables:

- Setpoints
- Differential values
- Status flags
- Timers and counters

Trend Visualization

Real-Time Charts

View live data trends:

1. Navigate to **Trend Logs** page
2. Select points to monitor

3. Choose time range (1 hour, 4 hours, 24 hours)
4. View interactive charts with zoom/pan

Chart Types:

- Line charts (analog values)
- Step charts (digital states)
- Multi-axis plots (different units)
- Stacked area charts

Historical Data

Access stored trend data:

- **Time Range Selection**: Custom date/time picker
- **Data Export**: Download CSV, Excel, JSON
- **Statistics**: Min, max, average, standard deviation
- **Annotations**: Add notes to specific timestamps

Alarm Monitoring

Active Alarms

View current alarm conditions:

Priority	Point	Message	Time	Acknowledge
High	IN1	High Temperature	10:23 AM	☐
Medium	OUT2	Fan Feedback Error	10:15 AM	☑

Alarm Actions:

- Acknowledge individual alarms
- Acknowledge all alarms
- Filter by priority/type
- View alarm history

Notification Settings

Configure alarm notifications:

- **Email Alerts**: Send to specified addresses
- **SMS Notifications**: Text message alerts

- **Audio Alerts**: Browser notification sounds
- **Popup Windows**: Desktop notifications

System Health

Performance Metrics

Monitor system performance:

```
TEXT

API Response Time:    45ms (avg)
Database Queries:     150/sec
Active Connections:   12
Memory Usage:         245MB / 512MB
CPU Load:            15%
```

Communication Stats

Track data collection:

- **Successful Polls**: Count of successful data requests
- **Failed Polls**: Error count and types
- **Average Response Time**: Communication latency
- **Retry Count**: Number of automatic retries

Auto-Refresh

Data automatically refreshes at configurable intervals:

- **Inputs/Outputs**: 5 seconds (default)
- **Variables**: 5 seconds (default)
- **Trends**: 10 seconds (real-time mode)
- **Alarms**: 3 seconds (high priority)

Manual Refresh: Click the 🔄 **Refresh from Device** button to force immediate update.

Filters and Search

Quick Filters

- **Status**: Show only online/offline/error devices
- **Type**: Filter by point type (AI, AO, AV, BI, BO, BV)
- **Alarm**: Show only alarmed points

- **Override**: Show only overridden points

Search

Use the search box to find specific points:

- Search by name, description, or ID
- Case-insensitive matching
- Wildcard support (*, ?)

Data Export

Export monitored data:

Point List Export

1. Select points to export
2. Click **Export** button
3. Choose format (CSV, Excel, JSON)
4. Save file

Trend Data Export

1. Set time range
2. Select points
3. Choose interval (1s, 5s, 1m, etc.)
4. Export with timestamps

Best Practices

1. **Set Appropriate Refresh Rates**: Balance responsiveness vs. network load
2. **Use Filters**: Focus on critical points
3. **Monitor Trends**: Watch for unusual patterns
4. **Review Alarms Daily**: Don't ignore persistent alarms
5. **Export Regular Reports**: Keep historical records
6. **Configure Thresholds**: Set appropriate alarm limits

Performance Tips

- Limit number of simultaneous trend charts (max 10)
- Use longer refresh intervals for non-critical points
- Archive old trend data regularly

- Close unused browser tabs
- Use modern browsers (Chrome, Edge, Firefox)

Next Steps

- Trend Logs - Detailed trend analysis
- Alarms - Alarm configuration
- Troubleshooting - Resolve monitoring issues

Device Monitoring

Real-Time Data Streaming

WebSocket Subscriptions

TYPESCRIPT

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

// Subscribe to specific points
ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'OUT1'],
    interval: 5000  // Update interval in ms
  }));
};

// Handle real-time updates
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  switch (data.type) {
    case 'point-update':
      updatePointValue(data.point, data.value, data.timestamp);
      break;
    case 'alarm':
      handleAlarm(data.alarm);
      break;
    case 'device-status':
      updateDeviceStatus(data.deviceId, data.status);
      break;
  }
};
```

REST API Polling

TYPESCRIPT

```typescript
class DataPoller {
  private intervals = new Map<string, NodeJS.Timeout>();

  async pollPoints(deviceId: number, points: string[], interval: number) {
    const pollerId = `${deviceId}:${points.join(',')}`;

    // Clear existing poller
    if (this.intervals.has(pollerId)) {
      clearInterval(this.intervals.get(pollerId)!);
    }

    // Start new poller
    const timer = setInterval(async () => {
      try {
        const response = await fetch(
          `http://localhost:9103/api/devices/${deviceId}/points?ids=${points.join(',')}`
        );
        const data = await response.json();

        data.points.forEach(point => {
          this.emit('update', point);
        });
      } catch (error) {
        this.emit('error', error);
      }
    }, interval);

    this.intervals.set(pollerId, timer);
  }

  stopPolling(deviceId: number, points: string[]) {
    const pollerId = `${deviceId}:${points.join(',')}`;
    const timer = this.intervals.get(pollerId);
    if (timer) {
      clearInterval(timer);
      this.intervals.delete(pollerId);
    }
```

```
    }
}
```

Data Aggregation and Caching

In-Memory Cache

TYPESCRIPT

```typescript
interface CachedPoint {
  value: number;
  timestamp: number;
  quality: string;
  ttl: number;
}

class PointCache {
  private cache = new Map<string, CachedPoint>();
  private readonly defaultTTL = 10000; // 10 seconds

  set(pointId: string, value: number, quality: string = 'good') {
    this.cache.set(pointId, {
      value,
      timestamp: Date.now(),
      quality,
      ttl: this.defaultTTL
    });
  }

  get(pointId: string): CachedPoint | null {
    const cached = this.cache.get(pointId);
    if (!cached) return null;

    // Check if expired
    if (Date.now() - cached.timestamp > cached.ttl) {
      this.cache.delete(pointId);
      return null;
    }

    return cached;
  }

  invalidate(pointId: string) {
    this.cache.delete(pointId);
  }
```

```
  clear() {
    this.cache.clear();
  }
}
```

Batch Reading Optimization

TYPESCRIPT

```
async function batchReadPoints(
  deviceId: number,
  points: string[],
  maxBatchSize: number = 50
): Promise<Map<string, number>> {
  const results = new Map<string, number>();

  // Split into batches
  for (let i = 0; i < points.length; i += maxBatchSize) {
    const batch = points.slice(i, i + maxBatchSize);

    const response = await fetch(
      `http://localhost:9103/api/devices/${deviceId}/points/batch`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ points: batch })
      }
    );

    const data = await response.json();
    data.values.forEach((value, index) => {
      results.set(batch[index], value);
    });
  }

  return results;
}
```

Trend Data Storage

SQLite Trend Table

SQL

```sql
CREATE TABLE trend_data (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER NOT NULL,
  point_id TEXT NOT NULL,
  value REAL NOT NULL,
  quality TEXT DEFAULT 'good',
  timestamp INTEGER NOT NULL,
  FOREIGN KEY (device_id) REFERENCES devices(id)
);

CREATE INDEX idx_trends_device_point
  ON trend_data(device_id, point_id);
CREATE INDEX idx_trends_timestamp
  ON trend_data(timestamp DESC);

-- Partitioning by time for better performance
CREATE INDEX idx_trends_time_device
  ON trend_data(timestamp DESC, device_id, point_id);
```

Trend Data Insertion

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class TrendLogger {
  private db: Database.Database;
  private insertStmt: Database.Statement;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.insertStmt = this.db.prepare(`
      INSERT INTO trend_data (device_id, point_id, value, quality, timestamp)
      VALUES (?, ?, ?, ?, ?)
    `);
  }

  log(deviceId: number, pointId: string, value: number, quality: string = 'good') {
    this.insertStmt.run(
      deviceId,
      pointId,
      value,
      quality,
      Date.now()
    );
  }

  logBatch(entries: Array<{deviceId: number, pointId: string, value: number}>) {
    const transaction = this.db.transaction((entries) => {
      for (const entry of entries) {
        this.insertStmt.run(
          entry.deviceId,
          entry.pointId,
          entry.value,
          'good',
          Date.now()
        );
      }
    });

    transaction(entries);
```

```
    }
}
```

Trend Data Query and Aggregation

TYPESCRIPT

```typescript
interface TrendQuery {
  deviceId: number;
  pointId: string;
  startTime: number;
  endTime: number;
  interval?: number;  // Aggregation interval in seconds
}

async function queryTrendData(query: TrendQuery) {
  const db = new Database('t3000.db');

  let sql: string;

  if (query.interval) {
    // Aggregated query
    sql = `
      SELECT
        (timestamp / ${query.interval * 1000}) * ${query.interval * 1000} as bucket,
        AVG(value) as avg_value,
        MIN(value) as min_value,
        MAX(value) as max_value,
        COUNT(*) as sample_count
      FROM trend_data
      WHERE device_id = ?
        AND point_id = ?
        AND timestamp BETWEEN ? AND ?
      GROUP BY bucket
      ORDER BY bucket
    `;
  } else {
    // Raw data query
    sql = `
      SELECT timestamp, value, quality
      FROM trend_data
      WHERE device_id = ?
        AND point_id = ?
        AND timestamp BETWEEN ? AND ?
      ORDER BY timestamp
```

```
    `;
  }

  return db.prepare(sql).all(
    query.deviceId,
    query.pointId,
    query.startTime,
    query.endTime
  );
}
```

Performance Monitoring

Metrics Collection

TYPESCRIPT

```typescript
class PerformanceMonitor {
  private metrics = {
    apiLatency: [] as number[],
    databaseQueries: 0,
    cacheHits: 0,
    cacheMisses: 0,
    activeConnections: 0
  };

  recordApiLatency(latency: number) {
    this.metrics.apiLatency.push(latency);
    if (this.metrics.apiLatency.length > 1000) {
      this.metrics.apiLatency.shift();
    }
  }

  getStats() {
    const latencies = this.metrics.apiLatency;
    return {
      avgLatency: latencies.reduce((a, b) => a + b, 0) / latencies.length,
      p95Latency: this.percentile(latencies, 0.95),
      p99Latency: this.percentile(latencies, 0.99),
      databaseQps: this.metrics.databaseQueries,
      cacheHitRate: this.metrics.cacheHits /
        (this.metrics.cacheHits + this.metrics.cacheMisses),
      activeConnections: this.metrics.activeConnections
    };
  }

  private percentile(arr: number[], p: number): number {
    const sorted = [...arr].sort((a, b) => a - b);
    const index = Math.ceil(sorted.length * p) - 1;
    return sorted[index];
  }
}
```

Automated Performance Optimization

TYPESCRIPT

```typescript
class AdaptivePolling {
  private currentInterval = 5000;
  private readonly minInterval = 1000;
  private readonly maxInterval = 30000;

  async adjustPollingRate(stats: PerformanceStats) {
    if (stats.avgLatency > 1000) {
      // High latency - reduce polling frequency
      this.currentInterval = Math.min(
        this.currentInterval * 1.5,
        this.maxInterval
      );
    } else if (stats.avgLatency < 200 && stats.cacheHitRate > 0.8) {
      // Good performance - can increase polling
      this.currentInterval = Math.max(
        this.currentInterval * 0.8,
        this.minInterval
      );
    }

    return this.currentInterval;
  }
}
```

Alarm Processing

Alarm Detection Engine

TYPESCRIPT

```typescript
interface AlarmRule {
  pointId: string;
  highLimit?: number;
  lowLimit?: number;
  deadband?: number;
  delay?: number;
  priority: 'high' | 'medium' | 'low';
}

class AlarmDetector {
  private rules = new Map<string, AlarmRule>();
  private states = new Map<string, {inAlarm: boolean, since: number}>();

  checkValue(pointId: string, value: number) {
    const rule = this.rules.get(pointId);
    if (!rule) return;

    let shouldAlarm = false;
    let message = '';

    if (rule.highLimit !== undefined && value > rule.highLimit) {
      shouldAlarm = true;
      message = `High limit exceeded: ${value} > ${rule.highLimit}`;
    } else if (rule.lowLimit !== undefined && value < rule.lowLimit) {
      shouldAlarm = true;
      message = `Low limit exceeded: ${value} < ${rule.lowLimit}`;
    }

    const state = this.states.get(pointId) || {inAlarm: false, since: 0};

    if (shouldAlarm && !state.inAlarm) {
      // New alarm
      const now = Date.now();
      if (rule.delay) {
        // Check if alarm persisted long enough
        if (!state.since) {
```

```
            state.since = now;
          } else if (now - state.since >= rule.delay * 1000) {
            this.triggerAlarm(pointId, message, rule.priority);
            state.inAlarm = true;
          }
        } else {
          this.triggerAlarm(pointId, message, rule.priority);
          state.inAlarm = true;
        }
      } else if (!shouldAlarm && state.inAlarm) {
        // Alarm cleared
        this.clearAlarm(pointId);
        state.inAlarm = false;
        state.since = 0;
      }

      this.states.set(pointId, state);
    }

    private triggerAlarm(pointId: string, message: string, priority: string) {
      // Store alarm in database
      // Send notifications
      // Emit event
    }

    private clearAlarm(pointId: string) {
      // Update alarm status
      // Send notification
    }
  }
```

Next Steps

- REST API Reference
- WebSocket API
- Performance Tuning
- Troubleshooting - Resolve monitoring issues